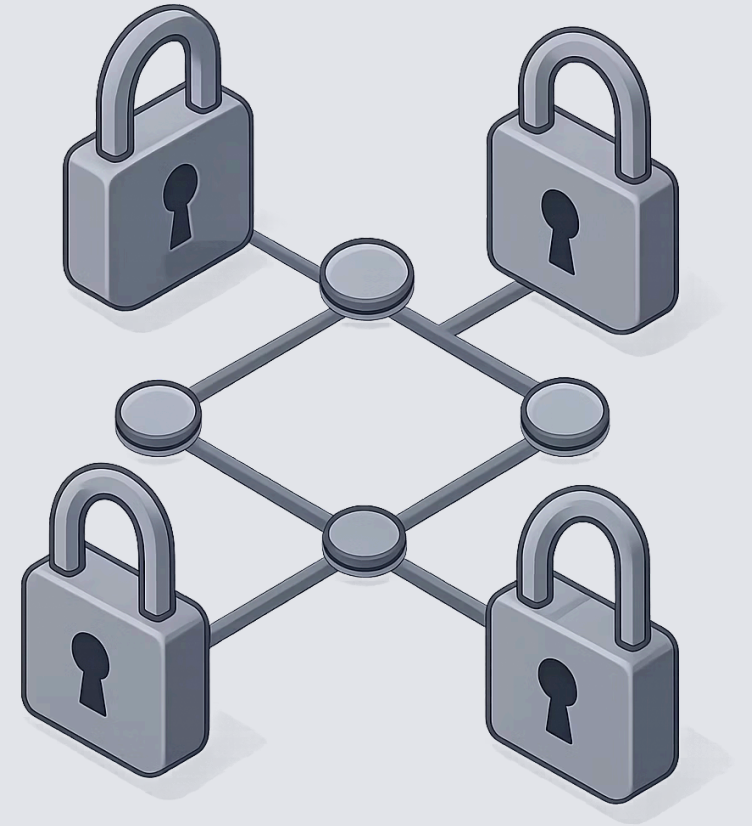


OAuth2 & OpenID Connect

Formation Spring – Protocoles d'authentification et d'autorisation modernes

Comprendre les fondements d'OAuth2 et OpenID Connect est essentiel pour tout développeur Spring. Ce guide vous accompagne pas à pas à travers les concepts clés, les flux d'autorisation les plus courants, et des exemples concrets avec GitHub et Google.



Au programme

Ce que vous allez apprendre

01

Les bases d'OAuth2

Rôles, tokens, scopes et le problème qu'OAuth2 résout

03

OpenID Connect

La couche d'identité au-dessus d'OAuth2 — ID Token et UserInfo

02

Les flux OAuth2

Authorization Code, Client Credentials, Implicit et Device Flow

04

Exemples concrets

Intégration avec GitHub et Google dans une application Spring

Le problème qu'OAuth2 résout

Avant OAuth2, pour accéder à vos données sur un service tiers, vous deviez partager votre **mot de passe** directement avec l'application. Ce modèle était dangereux : l'application avait un accès total, sans limite ni révocation possible.

✗ Avant OAuth2

- Partage du mot de passe à des tiers
- Accès illimité et irrévocable
- Aucune granularité des permissions
- Risque de compromission totale

✓ Avec OAuth2

- Délégation d'accès via un **token**
- Accès limité dans le temps et en portée
- Révocation possible à tout moment
- Le mot de passe n'est jamais partagé



Les acteurs d'OAuth2

OAuth2 définit quatre rôles distincts qui interagissent ensemble pour garantir un accès sécurisé aux ressources protégées.



Resource Owner

L'utilisateur final qui possède les données et qui donne son consentement explicite pour y accéder.



Client

L'application (web, mobile, serveur) qui souhaite accéder aux ressources au nom de l'utilisateur.



Resource Server

L'API ou le service qui héberge les ressources protégées et valide les tokens d'accès entrants.



Authorization Server

Le serveur qui authentifie l'utilisateur, obtient son consentement et émet les tokens (ex: Google, GitHub).

Les tokens OAuth2

OAuth2 utilise deux types de tokens complémentaires pour gérer l'accès aux ressources de façon sécurisée et flexible.

Access Token

Jeton de courte durée (quelques minutes à 1h) présenté à chaque requête API. Peut être un JWT ou un token opaque. Contient les **scopes** autorisés.

Refresh Token

Jeton de longue durée utilisé pour obtenir un nouvel Access Token sans redemander les identifiants. Stocké côté serveur, révocable à tout moment.

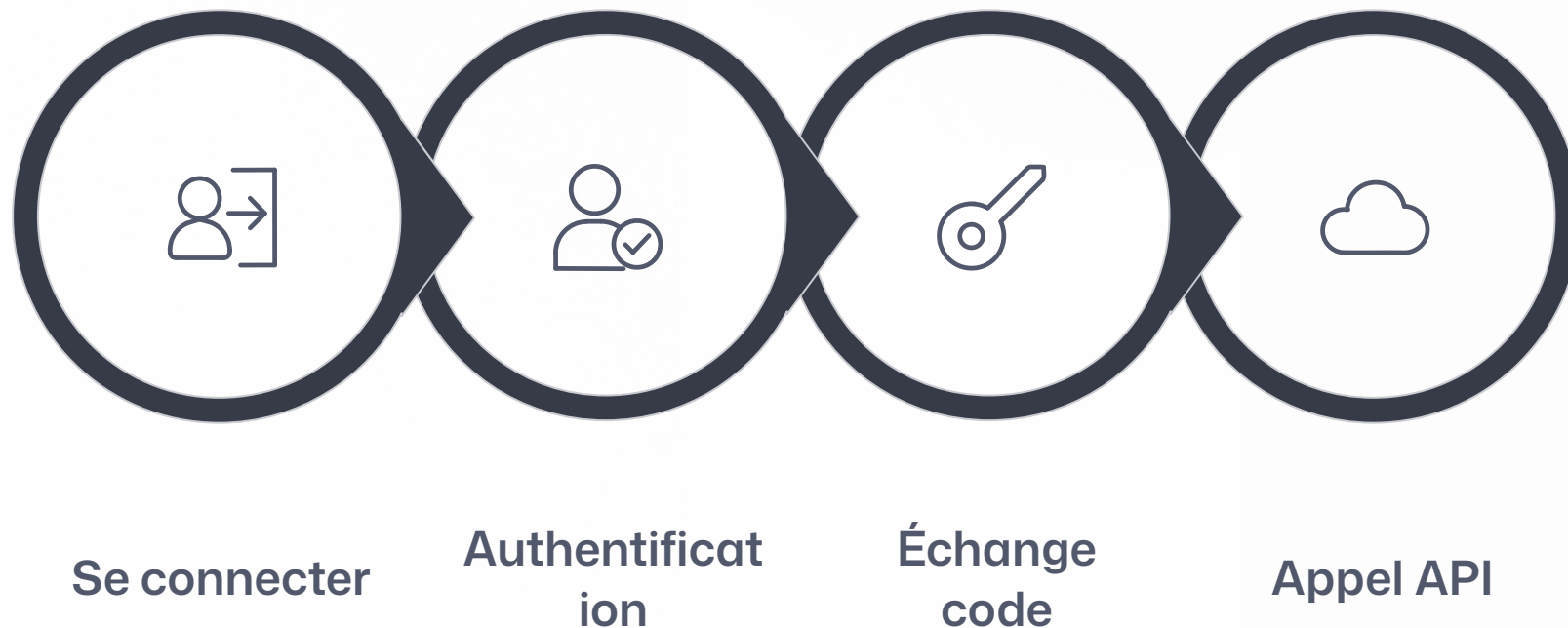
Exemple de JWT décodé

```
{  
  "sub": "1234567890",  
  "name": "Alice Dupont",  
  "scope": "read:profile email",  
  "iss": "https://auth.example.com",  
  "exp": 1716239022,  
  "iat": 1716235422  
}
```

Le champ `scope` liste les permissions accordées. Le champ `exp` indique la date d'expiration du token.

Authorization Code Flow

C'est le flux recommandé pour les applications web avec un backend. Il garantit que l'**Access Token** n'est jamais exposé dans le navigateur.



📌 **PKCE (Proof Key for Code Exchange)** : Extension obligatoire pour les apps publiques (SPA, mobile) afin d'empêcher l'interception du code d'autorisation.

Authorization Code Flow – Détail technique

Étape 1 – Redirection vers l'Authorization Server

```
GET https://accounts.google.com/o/oauth2/auth
?response_type=code
&client_id=YOUR_CLIENT_ID
&redirect_uri=https://app.com/callback
&scope=openid%20profile%20email
&state=xyz123
&code_challenge=E9Melhoa...
&code_challenge_method=S256
```

Étape 2 – Échange du code contre des tokens

```
POST https://oauth2.googleapis.com/token
grant_type=authorization_code
&code=AUTH_CODE
&redirect_uri=https://app.com/callback
&client_id=...
&client_secret=...
&code_verifier=...
```

Paramètres clés

response_type=code

Indique qu'on attend un code, pas un token directement

state

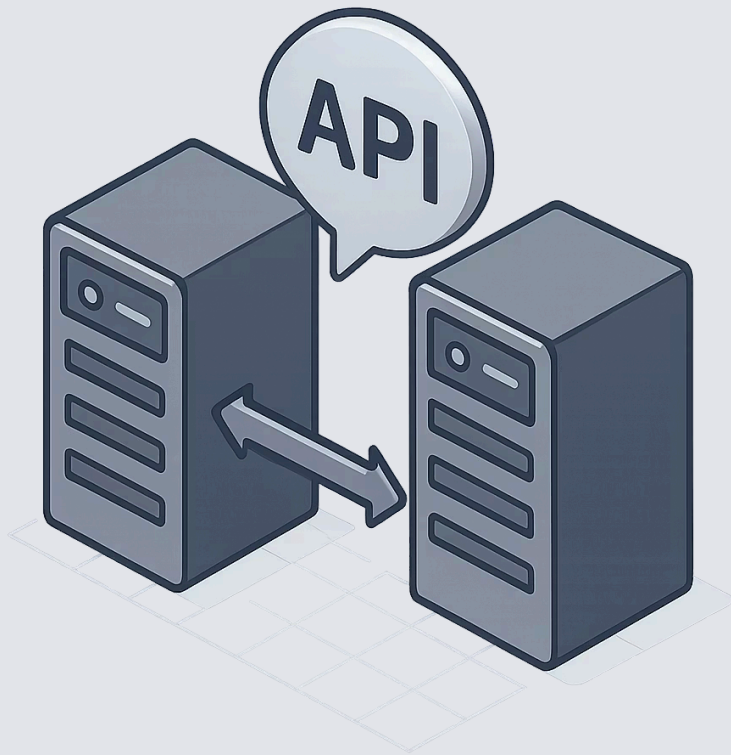
Valeur aléatoire anti-CSRF, vérifiée à la réception du callback

scope

Permissions demandées à l'utilisateur (ex: email profile)

code verifier / code_challenge

Mécanisme PKCE pour sécuriser le flux côté client



MACHINE TO MACHINE

Client Credentials Flow

Ce flux est utilisé pour les communications **serveur à serveur**, sans intervention d'un utilisateur humain. Le client s'authentifie directement avec son `client_id` et son `client_secret` pour obtenir un token d'accès.

Cas d'usage typiques

- Microservices qui s'appellent entre eux
- Tâches planifiées (cron jobs, batchs)
- Daemons et workers en arrière-plan

Requête token

```
POST /token
grant_type=client_credentials
&client_id=svc-billing
&client_secret=SECRET
&scope=invoices:read
```


Comparaison des flux OAuth2

Chaque flux est adapté à un contexte spécifique. Le choix du bon flux est crucial pour la sécurité de votre application.

Flux	Type d'app	Utilisateur	Sécurité
Authorization Code + PKCE	Web, SPA, Mobile	✓ Oui	★★★★ Très élevée
Client Credentials	Serveur / M2M	✗ Non	★★★★ Très élevée
Device Flow	TV, CLI, IoT	✓ Oui	★★★ Élevée
Implicit (déprécié)	SPA (ancienne pratique)	✓ Oui	★ Faible

OpenID Connect – La couche d'identité

OAuth2 seul

OAuth2 répond à la question : «**Puis-je accéder à cette ressource ?**»

Il délivre un Access Token pour autoriser des actions, mais ne dit *rien* sur l'identité de l'utilisateur.

- Autorisation 
- Authentification 
- Profil utilisateur 

OAuth2 + OpenID Connect

OIDC ajoute la réponse à : «**Qui est l'utilisateur ?**»

En ajoutant le scope `openid`, l'Authorization Server retourne également un **ID Token** (JWT) contenant l'identité.

- Autorisation 
- Authentification 
- Profil utilisateur 

L'ID Token – Le cœur d'OpenID Connect

L'ID Token est un **JWT signé** qui contient des informations standardisées sur l'utilisateur authentifié, appelées *claims*. Il est destiné à l'application cliente, pas à l'API.

Claims standards OIDC

```
{
  "iss": "https://accounts.google.com",
  "sub": "110169484474386276334",
  "aud": "my-client-id",
  "exp": 1716239022,
  "iat": 1716235422,
  "email": "alice@gmail.com",
  "name": "Alice Dupont",
  "picture": "https://...",
  "email_verified": true
}
```

Claims importants

sub (Subject)

Identifiant unique et stable de l'utilisateur chez le fournisseur

iss (Issuer)

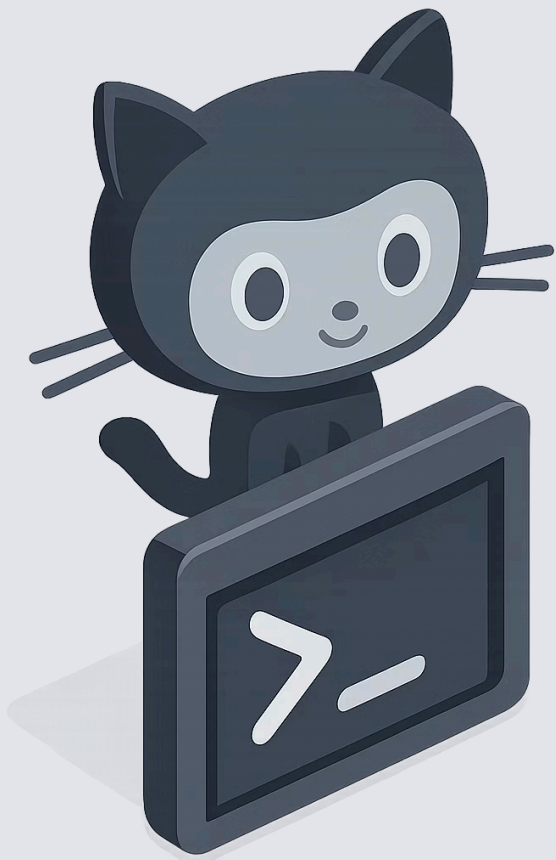
URL de l'Authorization Server qui a émis le token

aud (Audience)

Doit correspondre au `client_id` de votre application

Endpoint /userinfo

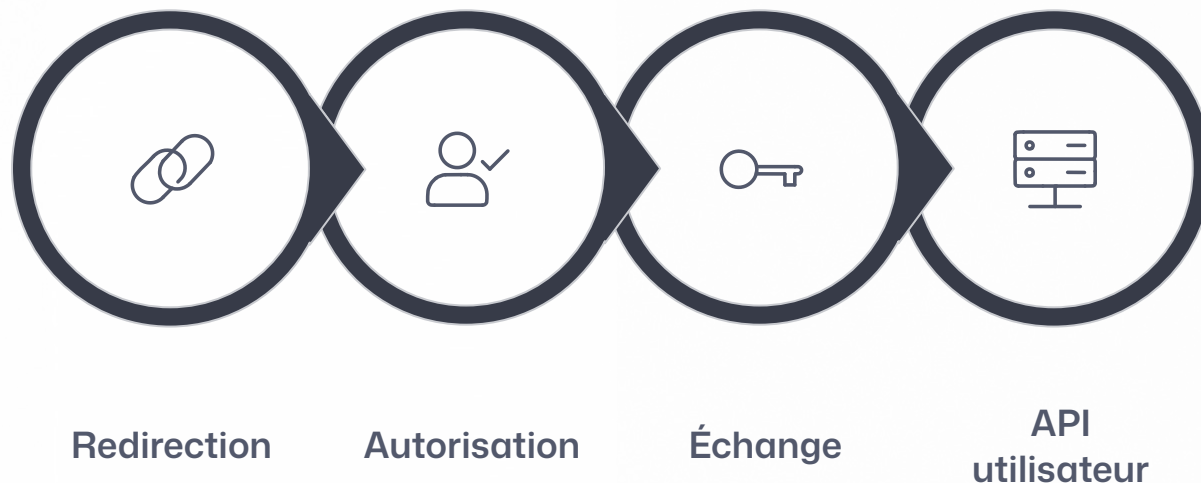
Permet de récupérer des claims supplémentaires avec l'Access Token



</> EXEMPLE CONCRET

OAuth2 avec GitHub

GitHub utilise le flux **Authorization Code** pour permettre à vos applications d'accéder aux dépôts, issues, et informations de profil de l'utilisateur.



Configuration Spring Boot – GitHub OAuth2

Spring Security rend l'intégration OAuth2/OIDC très simple grâce à son auto-configuration. Il suffit de quelques lignes pour activer le login social avec GitHub.

application.yml

```
spring:
  security:
    oauth2:
      client:
        registration:
          github:
            client-id: ${GITHUB_CLIENT_ID}
            client-secret: ${GITHUB_CLIENT_SECRET}
        scope:
          - read:user
          - user:email
```

Dépendance Maven

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Configuration SecurityFilterChain

```
@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(
        HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2Login(Customizer.withDefaults());
        return http.build();
    }
}
```

- ❑ Spring Security configure automatiquement les endpoints `/login/oauth2/code/github` comme callback URI.

OpenID Connect avec Google

Google est un fournisseur OIDC complet. En ajoutant le scope `openid`, votre application reçoit un **ID Token** avec les informations de profil, en plus de l'Access Token OAuth2.

application.yml avec Google

```
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id:
              ${GOOGLE_CLIENT_ID}
            client-secret:
              ${GOOGLE_CLIENT_SECRET}
            scope:
              - openid
              - profile
              - email
```

Récupérer l'utilisateur connecté

```
@GetMapping("/profile")
public Map<String, Object> profile(
    @AuthenticationPrincipal
    OidcUser oidcUser) {

    return Map.of(
        "name",
        oidcUser.getFullName(),
        "email", oidcUser.getEmail(),
        "sub", oidcUser.getSubject(),
        "pic", oidcUser.getPicture()
    );
}
```

Spring injecte automatiquement l'`OidcUser` contenant tous les claims de l'ID Token Google.



Points clés à retenir

OAuth2 = Autorisation

Délégation d'accès à des ressources via des tokens à durée limitée et révocables. Jamais de partage de mot de passe.

OIDC = Authentification

Couche d'identité au-dessus d'OAuth2. L'ID Token JWT prouve qui est l'utilisateur. Scope `openid` obligatoire.

Flux selon le contexte

Auth Code + PKCE pour les apps avec utilisateur. Client Credentials pour le M2M. Choisir le bon flux est essentiel.

Spring simplifie tout

Quelques lignes de `application.yml` suffisent pour intégrer GitHub ou Google dans une application Spring Boot.

OAuth2 n'est pas un protocole d'authentification — c'est OpenID Connect qui joue ce rôle. Ne jamais utiliser l'Access Token seul pour identifier un utilisateur.